

after which the *probe* function gets called.

Generic information

Probe in the driver gets called whenever a device's (already registered) name matches with the name of your platform driver (here, it is *bcm-gpio*). The second major functionality is *ioctl* which acts as a bridge between the application space and your driver. In technical terms, whenever your application invokes this (*ioctl*) system call, the call will be routed to this function of your driver. Once the call from the application is in your driver, you can process or provide data inside the driver and can respond to the application.

The SoC datasheet, i.e., BCM2835-ARM-Peripherals, plays a pivotal role in building up this driver. It consists of all the information pertaining to the peripherals supported by your SoC. It exposes all the registers relevant to a particular peripheral, which is where the key is. Once you know what registers of a peripheral are to be configured, half the job is done. Be cautious about which address has to be used to access these peripherals.

Types of addressing modes

There are three kinds of addressing modes - virtual addressing, physical addressing and system bus addressing. To learn the details, turn to Page 6 of the datasheet.

The macro `__io_address` implemented in the *probe* function of the driver returns the virtual address of the physical address passed as an argument. For GPIO, the physical address is `0x20200000(0x20000000 + 0x200000)`, where `0x20000000` is the base address and `0x200000` is the peripheral offset. Turn to Page 5 of the datasheet for more details. Any guesses on which address the macro `__io_address` would return? The address returned by this macro can then be used for accessing (reading or writing) the concerned peripheral registers.

The GPIO control application is analogous to a simple C program with an additional *ioctl* call. This call is capable of passing data from the application layer to the driver layer with an appropriate command. I have restricted the use of other GPIOs as they are not exposed to headers like others. So, modify the application as per your requirements. More information is available on this peripheral from Page 89 of the datasheet. In this code, I have just added functionality for setting or clearing a GPIO. Another interesting feature is that by configuring the appropriate registers, you can configure GPIOs as interrupt pins. So whenever a pulse is routed to that pin, the processor, i.e., ARM, is interrupted and the corresponding handler registered for that interrupt is invoked to handle and process it. This interesting aspect will be taken up in later articles.

Compilation of the GPIO device driver

There are two ways in which you can compile your driver.

- Cross compilation on the host PC

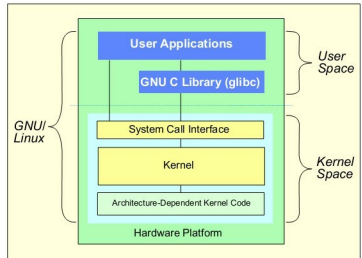


Figure 1: System layout



Figure 2: Console

- Local compilation on the target board
 - In the first method, one needs to have certain packages downloaded. These are:
 - ARM cross-compiler
 - Raspbian kernel source (the kernel version must match with the one running on your Pi; otherwise, the driver will not load onto the OS due to the version mismatch)
 - In the second method, one needs to install certain packages on Pi.
 - Go to the following link and follow the steps indicated: <http://stackoverflow.com/questions/20167411/how-to-compile-a-kernel-module-for-raspberry-pi>
 - Or, follow the third answer at this link, the starting line of which says, "Here are the steps I used to build the 'Hello World' kernel module on Raspbian."
- I went ahead with the second method as it was more straightforward.

Testing on your Raspberry Pi

Boot up your Raspberry Pi using minicom and you will see the console that resembles mine (Figure 2).